逻辑独立性：数据库变，应用层不变？

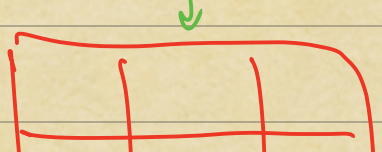
物理独立性：？

数据库：数据库的
- 数据模型：数据结构、数据操作、数据完整性约束
- 型：实体、属性、联系

概念模型：E-R

关系：
- 元组：一行
- 关系是笛卡尔集的子集
- 关系的码：可为实体，可作一？
- 属性满足实体律

关系：表

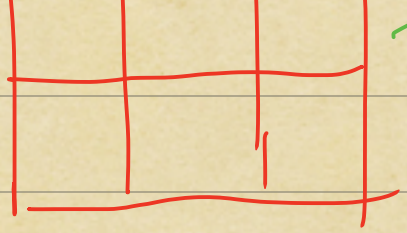


元组：1行

属性：一列

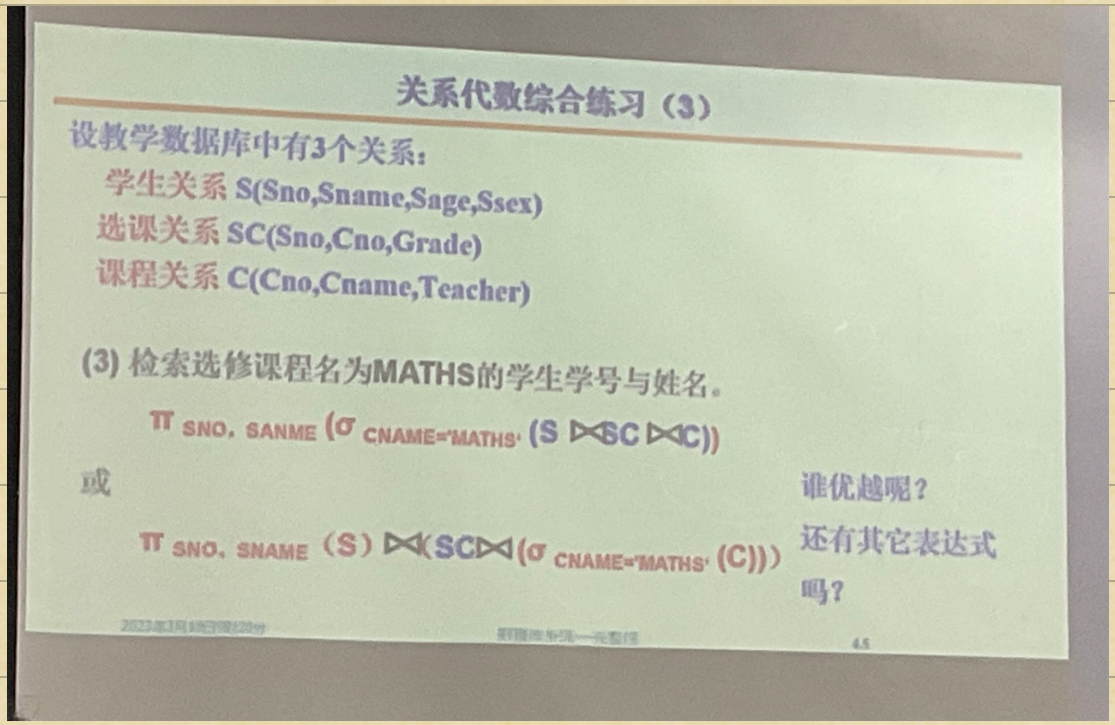
域：属性的取值

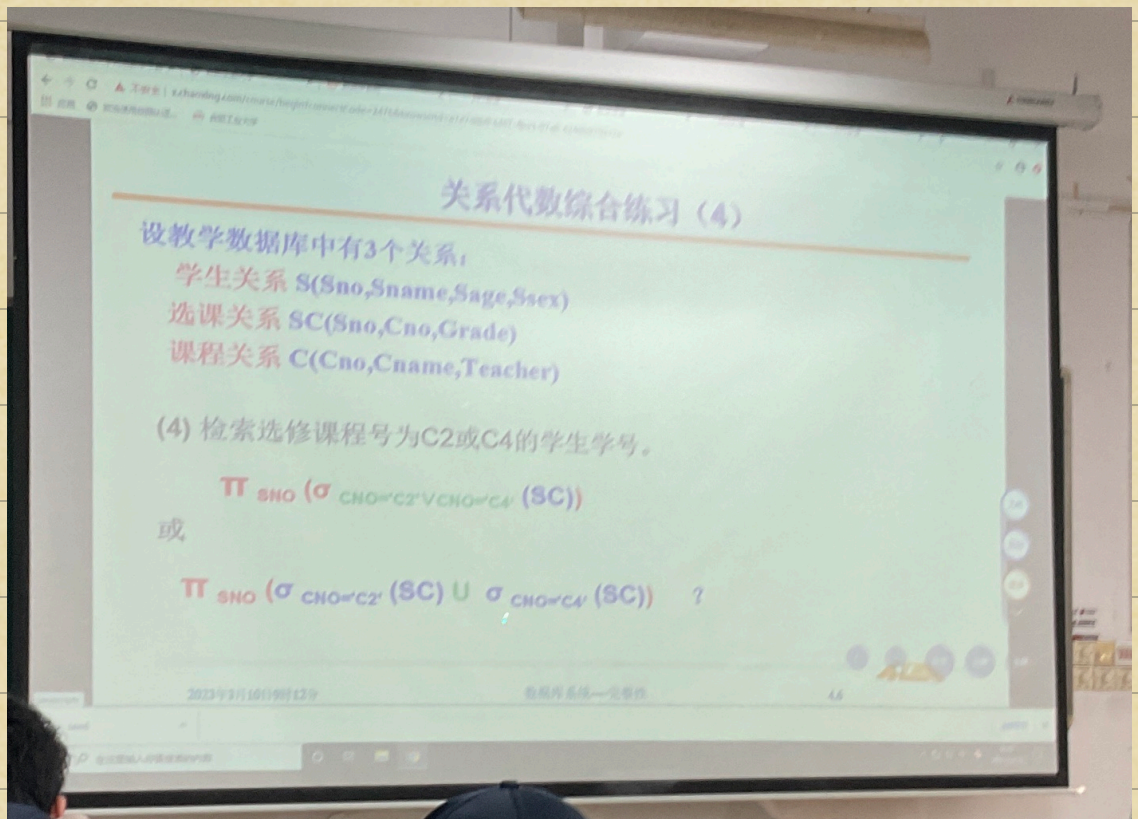
分量：元组中的一个属性值

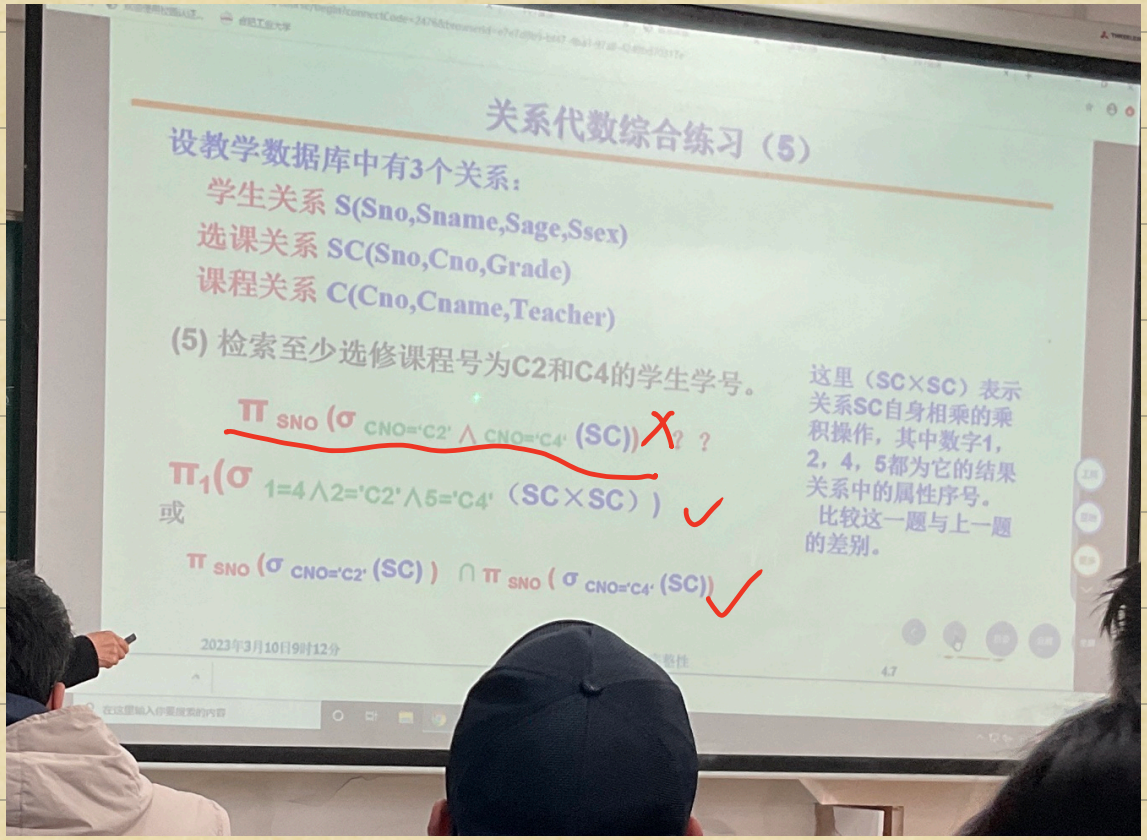


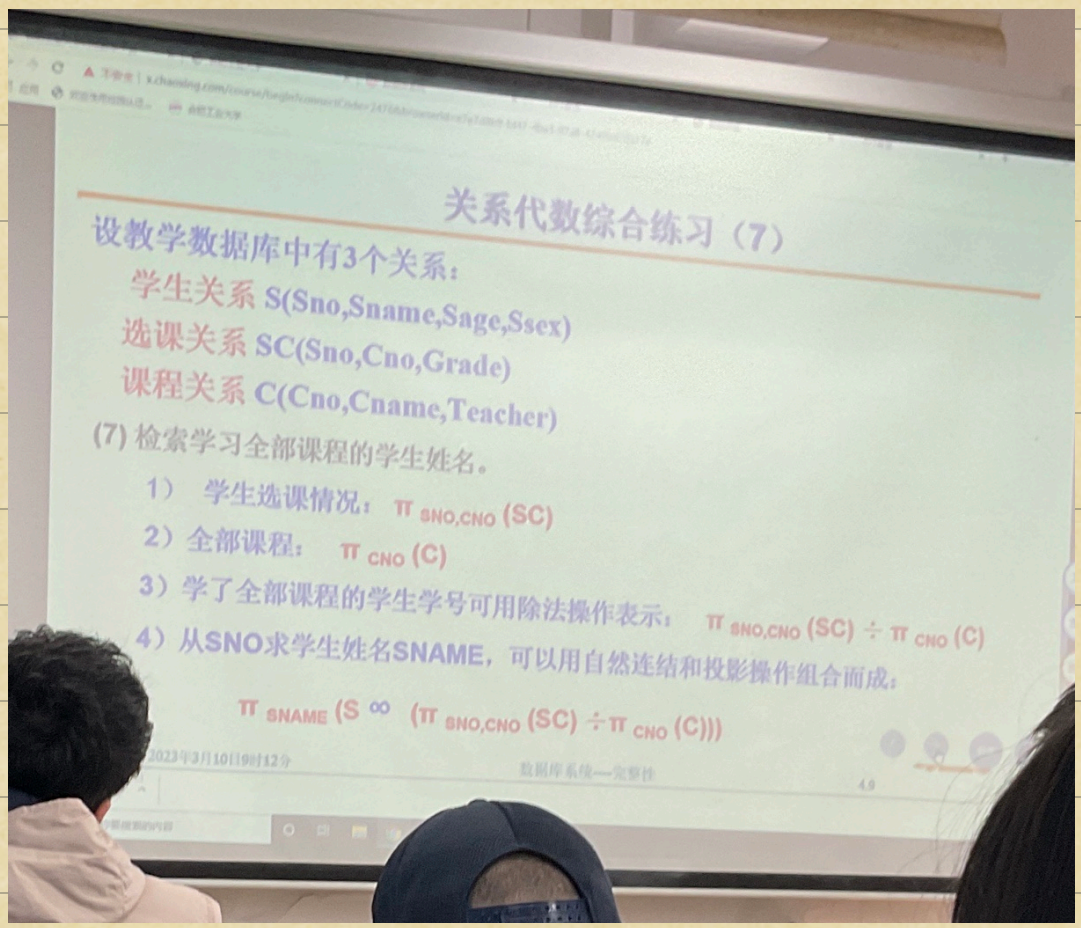
一个n元关系

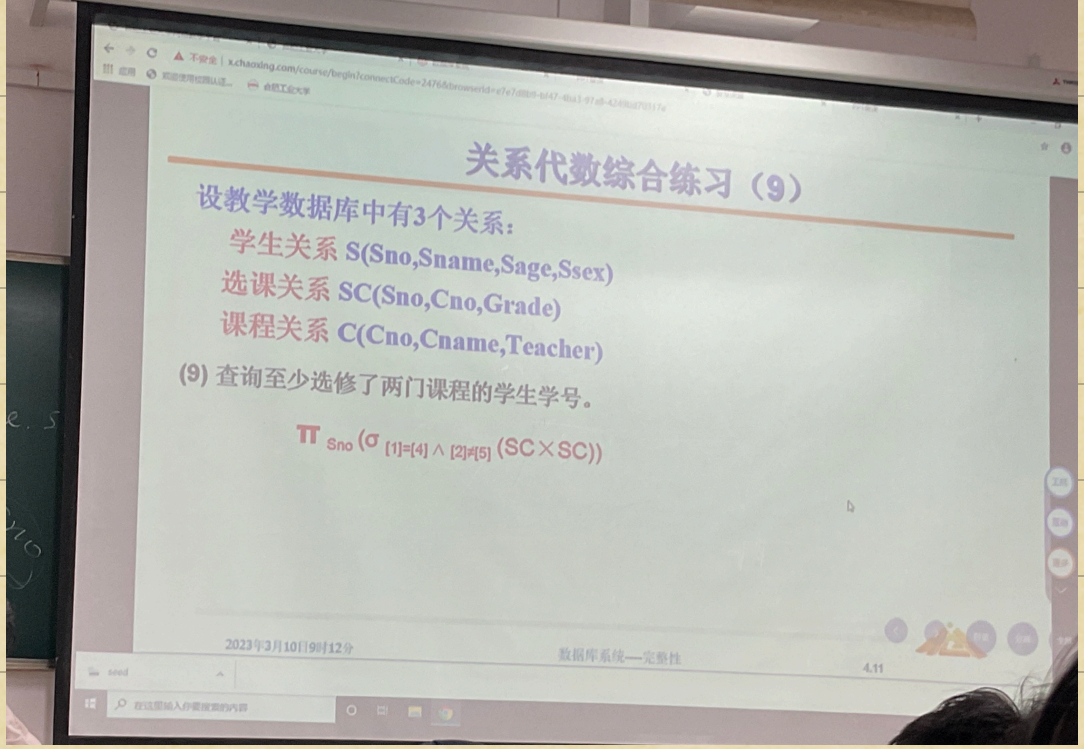
元组

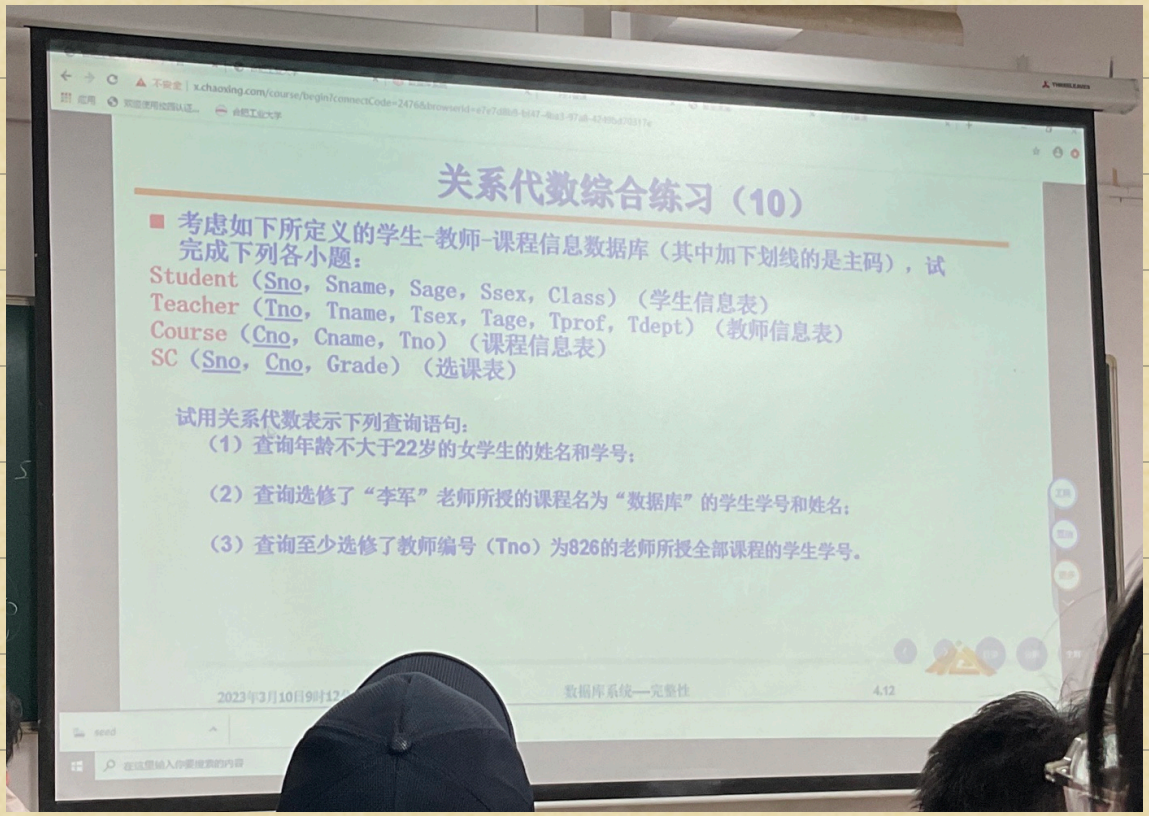





关系代数综合练习（5）
设教学数据库中有3个关系：
学生关系 S(Sno,Sname,Sage,Ssex)
选课关系 SC(Sno,Cno,Grade)
课程关系 C(Cno,Cname,Teacher)
(5) 检索至少选修课程号为C2和C4的学生学号。
这里（SC×SC）表示关系SC自身相乘的乘积操作，其中数字1，2，4，5都为它的结果关系中的属性序号。
比较这一题与上一题的差别。
或
2023年3月10日9时12分


关系代数综合练习（7）
设教学数据库中有3个关系：
学生关系 S(Sno,Sname,Sage,Ssex)
选课关系 SC(Sno,Cno,Grade)
课程关系 C(Cno,Cname,Teacher)
(7) 检索学习全部课程的学生姓名。
1） 学生选课情况：
2）全部课程：
3）学了全部课程的学生学号可用除法操作表示：
4）从SNO求学生姓名SNAME，可以用自然连结和投影操作组合而成：
2023年3月10日9时12分


关系代数综合练习（9）
设教学数据库中有3个关系：
学生关系 S(Sno,Sname,Sage,Ssex)
选课关系 SC(Sno,Cno,Grade)
课程关系 C(Cno,Cname,Teacher)
(9) 查询至少选修了两门课程的学生学号。
Π Sno (σ [1]=[4] ∧ [2]≠[5] (SC×SC))
2023年3月10日9时12分
数据库系统—完整性
4.11


关系代数综合练习（10）
考虑如下所定义的学生-教师-课程信息数据库（其中加下划线的是主码），试完成下列各小题：
Student（Sno，Sname，Sage，Ssex，Class）（学生信息表）
Teacher（Tno，Tname，Tsex，Tage，Tprof，Tdept）（教师信息表）
Course（Cno，Cname，Tno）（课程信息表）
SC（Sno，Cno，Grade）（选课表）
试用关系代数表示下列查询语句：
（1）查询年龄不大于22岁的女学生的姓名和学号；
（2）查询选修了“李军”老师所授的课程名为“数据库”的学生学号和姓名；
（3）查询至少选修了教师编号（Tno）为826的老师所授全部课程的学生学号。
数据库系统—完整性
4.12

第一章 绪论

(一) DB相关述

1. 4个基本概念

Data

Database

DBMS

DBS → 包括 { DB / DBMS / DBA (admin 管理员)

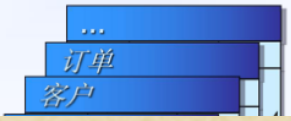
DBO: (owner)

2. 数据库定义

DB是长期储存在计算机内有组织的、可共享、大量的数据结合

3. 特点. (作业)

- 数据按一定的数据模型组织、描述和储存;
- 可为各种用户共享;
- 冗余度较小;
- 数据独立性较高;
- 易扩展



4. 特征

(1) DB是相互关联的数据的集合

(2) 用综合的方法组织数据 : 次序、索引、聚簇

(3) 用户共享,最小冗余

(4) 较高数据独立性:

数据独立性：包括物理独立性、逻辑独立性。
较高的独立性：数据库中的数据不仅可以给甲程序使用，还能给乙程序使用，且使用规则相同，即与使用程序无关。
指数据的组织和存储方法与应用程序互不依赖，彼此独立的特性。可降低应用程序的开发代价和维护代价。

(5) 安全机制. 数据安全可靠

(6) 允许并发

（6）允许并发地使用数据库，能有效、及时地处理数据，并能保证数据的一致性和完整性

一致性：数据库中的数据是共享的，并且允许多个用户同时使用相同的数据。这就要求数据库能够协议一致，保证各个用户之间对数据的操作不发生矛盾和冲突。

正确性、完整性：保证数据正确的特性——数据完整性

可通过建立一些约束条件保证数据库中的数据是正确的。

如：学生年龄20 （2或100则错误）

5. 什么是数据库

数据库是相互关联的数据的集合，并且它应该具有如下性质：用综合的方法组织数据，具有较小的数据冗余，可供多个用户共享，具有较高的数据独立性，具有安全控制机制，能够保证数据的安全、可靠，允许并发地使用数据库，能有效、及时地处理数据，并能保证数据的一致性和完整性。

6. DBMS

- 什么是DBMS
  - 数据库管理系统（Database Management System，简称DBMS）是位于用户与操作系统之间的一层数据管理软件。实现对数据库数据对象的统一管理、控制和维护。

- DBMS任务：

在数据库建立、运用和维护时对数据库进行统一控制，以保证数据的完整性、安全性、并在多用户同时使用数据库时进行并发控制，且在数据库系统发生故障后对系统进行恢复。

(作业)

DBMS功能：

1. **数据库定义功能：科学地组织和存储数据，通过提供数据定义语言（DDL）而实现**

定义数据库结构和存储结构；

定义数据库中数据之间的联系；

定义数据完整性约束条件和保证完整性的触发机制等.

2. **数据库操纵功能：高效地获取和维护数据，通过提供数据操纵语言（DML）而实现，**

完成对数据库中数据的操作：插入、删除、修改；

重新组织数据库的存储结构；

完成对数据库的备份/恢复等.

3. **数据库查询功能**

以各种方式提供灵活的查询功能，以便方便使用数据.

4. **数据库的运行管理**

保证数据的安全性、完整性、多用户对数据的并发使用、发生故障后的系统恢复。

5. **数据库的建立和维护功能(实用程序)**

数据库数据批量装载、数据库转储、介质故障恢复、数据库的重组织、性能监视等。

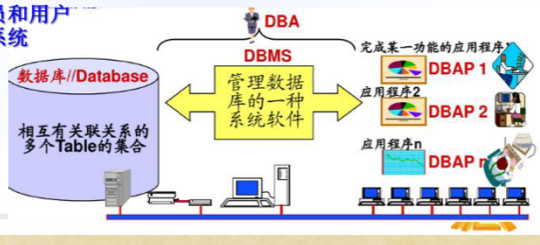
6. **数据库通信功能**

在分布式数据库或提供网络操作功能的数据库中还必须提供通信功能。

7. DBS

- 什么是数据库系统
  - 数据库系统（Database System，简称DBS）是指在计算机系统中引入数据库后的系统构成。
  - 在不引起混淆的情况下常常把数据库系统简称为数据库。

- **数据库系统的构成**
  - **数据库**
  - **数据库管理系统（及其开发工具）**
  - **应用系统**
  - **数据库管理员和用户**
  - **计算机基本系统**



8. 数据库管理技术三个阶段

人工管理阶段(40年代中--50年代中)

文件系统阶段(50年代末--60年代中) → 冗余、独立性低、一致性差

数据库系统阶段(60年代末--现在)

例：增加存储设备数据存储方式

应用程序与数据库数据相独立

9. DBS特点

- 数据的管理者：DBMS
- 数据面向的对象：现实世界
- 数据的共享程度：共享性高
- 数据的独立性：高度的物理独立性和一定的逻辑独立性
- 数据的结构化：整体结构化
- 数据控制能力：由DBMS统一管理和控制

应用程序与数据库逻辑结构独立

例：增加、删除一些列

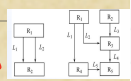
三要素

(二) 数据模型

作业

现实世界 → 概念模型 → 数据模型

(三) 数据库系统的结构



模式的3条

用户模型

只有1个

物理结构与存储方式

外模式 → 模式 → 内模式

(又叫用户模式)

↓ 映像 ↓

(保证外模式不变)

外模式/模式 逻辑独立性

(保证模式不变)

模式/内模式 物理独立性

(四) 数据库系统的组成

硬件、软件、数据库、人员

第二章. 关系数据库

(一) 结构及定义

1. 关系

二维表

$d=\{1,2\}$

$d_2=\{3,4,5\}$

✓ 域：一列（有相同数据类型）

笛卡尔积：全集；基数，域可值的乘积：2×3

✓ 元组：一行

分量：元组中的1个

属性：（学号，姓名，性别）

码 | 候选码：

> 若关系中的某一属性组的值能唯一地标识一个元组，而其子集不能，则称该属性组为候选码。
>
> 关系：Student(SNo，SName，SAge)

主码：

全码：全是候选码

外码：（学号 课号）→（课号，课名）

外码 → 主码

（当为一个关系的主码）

其属性为主属性.

2. 三类关系

基本关系

查询表

视图表

3. 关系模式

关系模式可以形式化地表示为：

R（U，D，DOM，F）

R	关系名
U	组成该关系的属性名集合
D	属性组U中属性所来自的域
DOM	属性向域的映象集合
F	属性间的数据依赖关系集合

(二) 关系操作

选择、投影、连接、除、并、交差

(三) 关系的完整性

必须满足 → 实体完整性：主属性不为空

参照完整性：外码的值为空或为对应主码域的值

用户定义完整性

参照关系：学生(学号, 专业号) → 引

被参照关系专业(专业号, 专业名)

Pd int not null

其值为空或在专业(专业号)中找到

(四) 关系代数

运算符		含义	运算符		含义
集合运算符	∪	并	比较运算符	>	大于
	-	差		≥	大于等于
	∩	交		<	小于
	×	广义笛卡尔积		≤	小于等于
				=	等于
				≠	不等于

表2.4 关系代数运算符（续）

运算符	含义		运算符	含义	
专门的关系运算符	σ	选择	逻辑运算符	¬	非
	Π	投影		∧	与
	⋈	连接		∨	或
	÷	除			

1. 并 (∪)

$R \cup S = \{t \mid t \in R \vee t \in S\}$ ：直接合并

2. 差：(-)

$R - S = \{t \mid t \in R \wedge t \notin S\}$

3. 交：(∩)

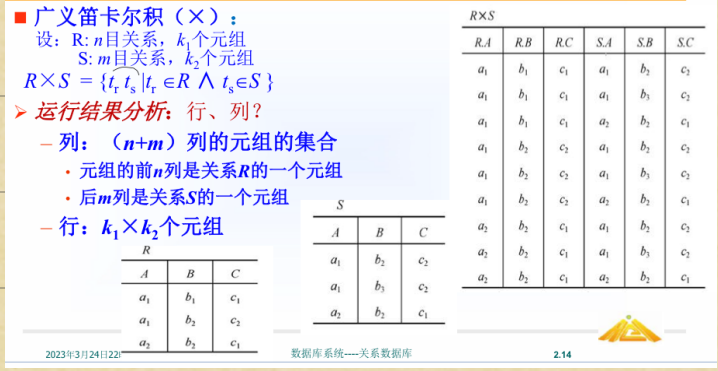
$R \cap S = \{t \mid t \in R \wedge t \in S\}$

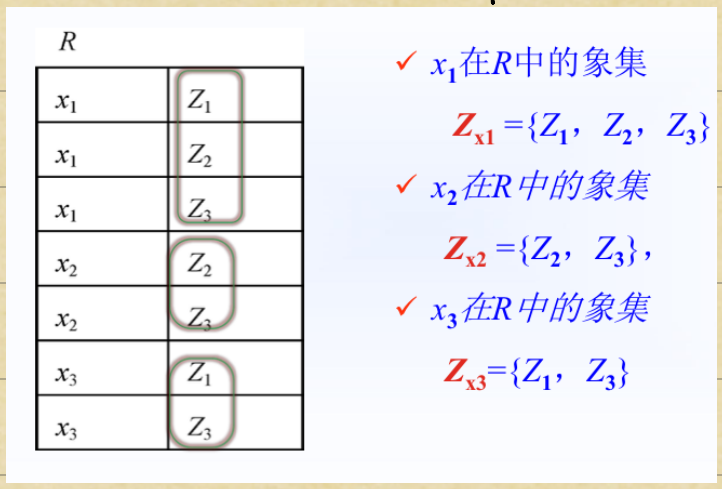
4. 笛卡尔集 (×)

$R \times S = \{t_r t_s \mid t_r \in R \wedge t_s \in S\}$

5. 象集 Z_x

$R(X, Z)$ $Z_x = \{t[Z] \mid t \in R, t[X] = x\}$





6. 选择（σ）

R(姓名, 学号) $\sigma_{学号='1234'}(R)$ ：选择R中的行

7. 投影（π）

$\pi_A(R) = \{t[A] \mid t \in R\}$ $\pi_{学号}(R)$：选择R的列 ：自动取消重复行

$\pi_2(R)$

8. 连接

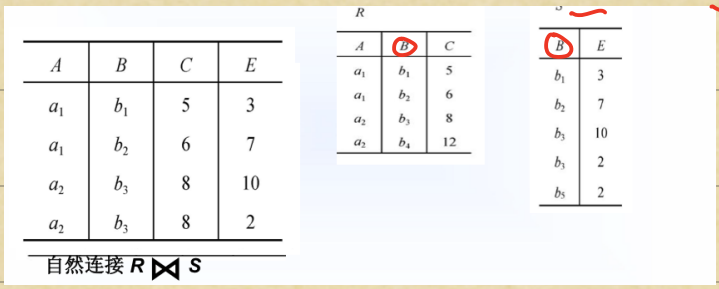
等值连接：$R \underset{A=B}{\bowtie} S = \{t_r t_s \mid t_r \in R \wedge t_s \in S \wedge t_r[A] = t_s[B]\}$

不同列

同一列

自然连接：$R \bowtie S = \{t_r t_s \mid t_r \in R \wedge t_s \in S \wedge t_r[B] = t_s[B]\}$

书P54



9. 悬浮元组

在一个连接中，若一个关系中的一个元祖不能和另外关系中的任何元组配对，则称该元组为悬浮元组（danging tuple）。

如果把舍弃的元组（**悬浮元组**）也保存在结果关系中，而在其他属性上填空值(Null)，这种连接就叫做外连接。

R

A	B	C
a_1	b_1	5
a_1	b_2	6
a_2	b_3	8
a_2	b_4	12

S

B	E
b_1	3
b_2	7
b_3	10
b_3	2
b_5	2

$R \bowtie S$

A	B	C	E
a_1	b_1	5	3
a_1	b_2	6	7
a_2	b_3	8	10
a_2	b_3	8	2
a_2	b_4	12	NULL
NULL	b_5	NULL	2

左外连接

如果只把左边关系R中的**悬浮元组**保留就叫做左外连接。

右外连接

如果只把右边关系S中的**悬浮元组**保留就叫做右外连接。

10 除（÷）

$R \div S = \{tr[X] \mid tr \in R \wedge \Pi_Y(S) \subseteq Y_x\}$

元组在X上的分量值x的象集Y_x包括S在Y的投影

设关系R、S分别为下图的(a)和(b)，$R \div S$的结果为图(c)

R

A	B	C
a_1	b_1	c_2
a_2	b_3	c_7
a_3	b_4	c_6
a_1	b_2	c_3
a_4	b_6	c_6
a_2	b_2	c_3
a_1	b_2	c_1

(a)

S

B	C	D
b_1	c_2	d_1
b_2	c_1	d_1
b_2	c_3	d_2

(b)

$R \div S$

A
a_1

(c)

- 它的含义是：A的象集至少包含 (b_1, c_2)、(b_2, c_1)、(b_2, c_3) 。

✓ 在关系R中，A可以取四个值{a_1，a_2，a_3，a_4}

a_1的象集为 {(b_1，c_2)，(b_2，c_3)，(b_2，c_1)}

a_2的象集为 {(b_3，c_7)，(b_2，c_3)}

a_3的象集为 {(b_4，c_6)}

a_4的象集为 {(b_6，c_6)}

投影到元素

✓ S在(B，C)上的投影为:

{(b_1，c_2)，(b_2，c_1)，(b_2，c_3)}

✓ 只有a1的象集包含了S在(B，C)属性组上的投影, 所以:

$R \div S = \{a_1\}$

第三章. SQL概述

(一)概述

DDL. DML DCL

Data Language

define manage control → Grant, Revoke

Create. Alter, Drop Insert Delete Update, Select

特点. CREATE/ALTER/DROP DataBASE/TABLE ...

(二)数据定义 定义.删除.改

1. Create

CREATE/DROP DATABASE 数据库名; USE 名.

2. 表的定义

- char (n) :固定长度的字符串
- varchar (n) :可变长字符串
- int :整数 // 有时不同系统也写作integer
- numeric (p, q) :固定精度数字，小数点左边p位，右边p-q位
- real :浮点精度数字 //有时不同系统也写作float(n)，小数点后保留n位
- date :日期（如 2003-09-12）
- time : 时间（如 23:15:003）
- …
- 现行商用DBMS的数据类型有时和上面有些差异，请注意；
  - ✓ 和高级语言的数据类型，总体上是一致的，但也有些差异。
  - ✓ 不同的DBMS其支持的数据类型也有差异。

书P83表

CREATE TABLE 表名(

Sno char(9) PRIMARY KEY,

Sname VARCHAR(20) UNIQUE);

> 区别 primary key = unique + not null

```
CREATE TABLE  Course
      ( Cno      CHAR(4)  PRIMARY KEY,
        Cname  CHAR(40),
        Cpno     CHAR(4) ,            // Cpno是先修课
        Credit  number(3,1) ,
       FOREIGN KEY (Cpno) REFERENCES  Course(Cno)
    );
```

// 定义Cpno为外码，被参照表是Course，被参照列是Cno。

定义约束.

3. 修改表

Alter table ... ADD column ...

drop column ...

modify column ...

```
ALTER TABLE <表名>
  [ ADD [COLUMN]<新列名> <数据类型> [ 完整性约束 ] ]
  [ADD <表级完整性约束>]
  [DROP [COLUMN] <列名>  [CASCADE| RESTRICT] ];
  [ DROP  CONSTRAINT<完整性约束名> [ RESTRICT | CASCADE ] ]
  [ ALTER COLUMN<列名> <数据类型> ];
```

[例8]：

```
ALTER TABLE Student ADD S_entrance DATE;
ALTER TABLE Student ALTER COLUMN Sage INT;
ALTER TABLE Student DROP COLUMN S_entrance;
ALTER TABLE Course ADD UNIQUE(Cname);
```

- 不论基本表中原来是否已有数据，新增加的列一律为空值
- 修改原有的列定义有可能会破坏已有数据

■ 删除基本表

```
DROP TABLE <表名>［RESTRICT| CASCADE］;
```

☞**RESTRICT**：删除表是有限制的。

- ✓欲删除的基本表不能被其他表的约束所引用
- ✓如果存在依赖该表的对象，则此表不能被删除

☞**CASCADE**：删除该表没有限制。

- ✓在删除基本表的同时，相关的依赖对象一起删除

[例9]：

```
DROP TABLE  Student  CASCADE ;
DROP TABLE  Student  RESTRICT;     // 需参照规定！
```

-> 连删

若有相关引用，不删除（默认）

4. 索引的建立

```
CREATE  [UNIQUE] [CLUSTER]  INDEX  <索引名>
        ON <表名>(<列名>[<次序>][,<列名>[<次序>] ]...);
```

- UNIQUE：此索引的每一个索引值只对应唯一的数据记录
- CLUSTER：表示要建立的索引是聚簇索引

*注意：*一个基本表上最多只能建立一个聚簇索引（概念见7.5.2节）。

[例14]：

```
CREATE CLUSTER INDEX Stusname   ON   Student(Sname);
CREATE UNIQUE INDEX  Stusno ON Student(Sno);
CREATE UNIQUE INDEX  Coucno ON Course(Cno);
CREATE UNIQUE INDEX  SCno ON SC(Sno ASC，Cno DESC);
```

■ 修改索引

ALTER INDEX <旧索引名> RENAME TO <新索引名>

[例15] 将SC表的SCno索引名改为SCSno

ALTER INDEX SCno RENAME TO SCSno;

■ 删除索引

DROP INDEX <索引名>;

删除索引时，系统会从数据字典中删去有关该索引的描述。

[例16] 删除Student表的Stusname索引

DROP INDEX Stusname;

(四)数据查询

- SELECT子句：指定要显示的属性列
- FROM子句：指定查询对象（基本表或视图）
- WHERE子句：指定查询条件

GROUPBY子句：对查询结果按指定列的值分组，该属性列值相等的元组为一个组。通常会在
作用聚集函数。

HAVING短语：只有满足指定条件的组才予以输出

ORDER BY子句：对查询结果表按指定列值的升序或降序排序

SELECT ALL | DISTINCT Sno FROM SC;

默认：ALL

Sno
201215121
201215121
201215121
201215122
201215122

Sno
201215121
201215122

WHERE子句常用的查询条件

查询条件	谓　词
比　较	=，>，<，>=，<=，!=，<>，!>，!<；NOT+上述比较运算符
确定范围	BETWEEN AND，NOT BETWEEN AND
确定集合	IN，NOT IN
字符匹配	LIKE，NOT LIKE
空　值	IS NULL，IS NOT NULL
多重条件（逻辑运算）	AND，OR，NOT

■ 谓词：IN <值表>, NOT IN <值表>

[例11] 查询信息系（IS）、数学系（MA）和计算机系（CS）学生的姓名和性别。

```
SELECT Sname, Ssex FROM Student
    WHERE Sdept IN ( 'IS'，'MA'，'CS' );
```

■ 字符串匹配： [NOT] LIKE '<匹配串>' [ESCAPE ' <换码字符>']

✓ <匹配串>：指定匹配模板（固定字符串或含通配符的字符串）

■ 通配符

◆ % (百分号) 代表任意长度（长度可以为0）的字符串

例：a%b表示以a开头，以b结尾的任意长度的字符串。如acb，addgb，ab 等都满足该匹配串

✧ _ (下横线) 代表任意单个字符

例：a_b表示以a开头，以b结尾的长度为3的任意字符串。如acb，afb等都满足该匹配串

■ ESCAPE 短语：当用户要查询的字符串本身就含有 % 或 _ 时，要使用ESCAPE '<换码字符>' 短语对通配符进行转义。

[例] SELECT Cno,Cname,Credit FROM Course
WHERE Cname LIKE '人工%' ;

Cno	Cname	Credit
10	人工智能	2
9	人工%Design	2

[例] SELECT Cno,Cname,Credit FROM Course
WHERE Cname LIKE '人工\%%';

Cno	Cname	Credit
9	人工%Design	2

但： SELECT Cno,Cname,Credit FROM Course WHERE Cname
LIKE '人工\%%' ESCAPE '\';

```
错误代码： 1064
You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near
```

必需是 { is null / is not null　　不能是 { = null / = not null

使用ORDER BY子句

- 可以按一个或多个属性列排序
- 升序：ASC；降序：DESC；缺省值为升序

➢ 当排序列含空值时

空值的显示顺序由具体系统实现来决定

例如：按升序排，含空值的元组最后显示

按降序排，空值的元组最先显示

➢ 各个系统的实现可以不同，只要保持一致即可

◆ 基于统计的查询（聚集函数）

- ➢ 统计元组个数

  COUNT(*)

- ➢ 统计一列中值的个数

  COUNT([DISTINCT|ALL] <列名>)

- ➢ 计算一列值的总和（此列必须为数值型）

  SUM([DISTINCT|ALL] <列名>)

- ➢ 计算一列值的平均值（此列必须为数值型）

  AVG([DISTINCT|ALL] <列名>)

- ➢ 求一列中的最大值和最小值

  MAX([DISTINCT|ALL] <列名>)

  MIN([DISTINCT|ALL] <列名>)

◆ 基于分组的统计查询（GROUP BY子句）

- ✓ 细化聚集函数的作用对象；
- ✓ 作用对象是查询的中间结果表；
- ✓ 按指定的一列或多列值分组，值相等的为一组。

[例24] 求各个课程号及相应的选课人数。

```
SELECT Cno，COUNT(Sno) FROM SC GROUP BY Cno；
```

Cno	COUNT(Sno)
1	2
2	2
3	2

对查询结果分组 （续）

- 使用GROUP BY子句后，SELECT子句的列名列表中只能出现分组属性和聚集函数，或者说，select 后面的所有列中,没有使用聚合函数的列,必须出现在 group by 后面

[例25] 查询选修了3门以上课程的学生学号。

```
SELECT Sno FROM SC GROUP BY Sno HAVING COUNT(*) >2；
```

Sno
202215121

- 只有满足HAVING短语指定条件的组才输出
- HAVING短语与WHERE子句的区别：作用对象不同
  - ✓ WHERE子句作用于基表或视图，从中选择满足条件的元组。
  - ✓ HAVING短语作用于组，从中选择满足条件的组。

■ [例3.48]查询平均成绩大于等于90分的学生学号和平均成绩

➢ 下面的语句是不对的：

```
SELECT Sno, AVG(Grade)
FROM SC
WHERE AVG(Grade)>=90
GROUP BY Sno;
```

```
1 queries executed, 0 success, 1 errors, 0 warnings
查询：SELECT Sno,AVG(Grade) FROM SC WHERE AVG(Grade)>=90 GROUP BY Sno LI
错误代码： 1111
Invalid use of group function
```

➢ 因为WHERE子句中是不能用聚集函数作为条件表达式

➢ 正确的查询语句应该是：

```
SELECT Sno, AVG(Grade)
FROM SC
GROUP BY Sno
HAVING AVG(Grade)>=88;
```

Sno	AVG(Grade)
202215121	88.3333

连接查询 { 等值与非等值 / 自身连接 / 外连接 / 多表连接

等值连接：不去重复列

自然连接：把重复列去掉

为 { 基于嵌套循环法 / 排序合并法 / 索引连接

自身连接：

■ 自身连接

✓ 一个表与其自己进行连接，称为表的**自身连接**

✓ 需要给表起别名以示区别

✓ 由于所有属性名都是同名属性，因此必须使用别名前缀

Second.Cpno → Second

[例28]查询每一门课的间接先修课（即先修课的先修课）。

```
SELECT FIRST.Cno，SECOND.Cpno
    FROM Course FIRST，Course SECOND
    WHERE FIRST.Cpno = SECOND.Cno;
```

(inner) join = where xx : xx

[例30] 改写[例26查询每个学生及其选修课程的情况]（包括没有选修课程的学生----用外连接操作）

```
SELECT Student.Sno，Sname，Ssex，Sage，Sdept，Cno，Grade
    FROM Student LEFT JOIN SC ON Student.Sno=SC.Sno;
```

连接类型：inner join / left outer join / right outer join / full outer join

默认inner 返回2表都在的行

以它为准

连接条件：natural / on <predicate> / using (A_1, A_2, ..., A_n)

Sno	Sname	Ssex	Sage	Sdept	Cno	Grade
202215121	李勇	男	20	CS	3	88
202215121	李勇	男	20	CS	2	85
202215121	李勇	男	20	CS	1	92
202215122	刘晨	女	19	CS	3	80
202215122	刘晨	女	19	CS	2	90
202215123	王敏	女	18	MA	(NULL)	(NULL)
202215125	张立	男	19	IS	(NULL)	(NULL)
202215126	欧阳杰	男	18	AI	1	(NULL)

3. 外连接

■ 外连接与普通连接的区别

➢ 普通连接操作只输出满足连接条件的元组

➢ 外连接操作以指定表为连接主体，将主体表中不满足连接条件的元组一并输出（不满足为null）

➢ 左外连接

列出左边关系中所有的元组

➢ 右外连接

列出右边关系中所有的元组

引连接.

多表：FROM 后面多表

嵌套查询.

```
SELECT Sname                    外层查询/父查询
FROM  Student
WHERE Sno IN
   （SELECT Sno                 内层查询/子查询
     FROM SC
     WHERE Cno= ' 2 '）；
```

- 一个SELECT-FROM-WHERE语句称为一个查询块。
- 将一个查询块嵌套在另一个查询块的WHERE子句或HAVING短语的条件中，称为子查询。
- 子查询中不能使用ORDER BY子句。

- **不相关子查询**

  子查询的查询条件不依赖于父查询。是由里向外逐层处理。即每个子查询在上一级查询处理之前求解，子查询的结果用于建立其父查询的查找条件。

- **相关子查询**

  子查询的查询条件依赖于父查询

  - 首先取外层查询中表的第一个元组，根据它与内层查询相关的属性值处理内层查询，若**WHERE**子句返回值为真，则取此元组放入结果表；
  - 然后再取外层表的下一个元组；
  - 重复这一过程，直至外层表全部检查完为止。

子查询：{ IN
比较运算符
ANY(任一) ALL(全部)
EXISTS

[例36] 查询其他系中比信息系所有学生年龄都小的学生姓名及年龄。

方法一：用ALL谓词

```
SELECT Sname，Sage   FROM Student
   WHERE Sage < ALL  (SELECT Sage  FROM Student WHERE Sdept= ' IS ')
         AND Sdept <> ' IS ';
```

方法二：用聚集函数

```
SELECT Sname，Sage   FROM Student
   WHERE Sage < (SELECT MIN(Sage)  FROM Student WHERE Sdept= ' IS ')
         AND Sdept <>' IS ';
```

[例38] 查询没有选修2号课程的学生姓名。

```
SELECT Sname   FROM  Student
   WHERE  NOT  EXISTS
     (SELECT *   FROM  SC
          WHERE Sno = Student.Sno AND Cno='2');
```

集合查询：Union，intersect，Except

并　　交　　差

[例41] 查询计算机科学系的学生及年龄不大于19岁的学生。

方法一：

```
SELECT * FROM Student WHERE Sdept= 'CS'
UNION
SELECT * FROM Student WHERE Sage<=19;
```

- UNION：将多个查询结果合并起来时，系统自动去掉重复元组。
- UNION ALL：将多个查询结果合并起来时，保留重复元组

[例42] 查询同时选修了课程1和课程3的学生学号。

方法一：集合查询

```
SELECT Sno FROM SC WHERE Cno=' 1 '
INTERSECT
SELECT Sno FROM SC WHERE Cno= '3 ';
```

[例43] 查询没有选修了课程1的学生学号。

方法一：集合查询

```
SELECT Sno FROM student
EXCEPT
SELECT Sno FROM SC WHERE Cno = '1';
```

基于派生表

- [例44] 找出每个学生超过他自己选修课程平均成绩的课程号

```
方法一：Select Sno, Cno From SC x
Where Grade >= (select avg(Grade)
                from SC y where y.Sno=x.Sno);
```

(五) 数据更新

插入数据

- insert into 表 values ...
- update 表 set ...
- delete from 表

```
insert into Student(Sno) values ('2020');
update Student set age=20 where Sno='2020';
delete from SC where Sno='2020'
```

(六) 空值

(七) 视图

create view 视图名 As 子查询 with check option

不允许 orderby 和 distinct

[例1] 建立信息系学生的视图。

```
CREATE VIEW IS_Student
    AS
    SELECT Sno，Sname，Sage FROM Student
        WHERE Sdept= 'IS'；
```

[例2] 建立信息系学生的视图，并要求进行修改和插入操作时仍需保证该视图只有信息系的学生。

```
CREATE VIEW IS_Student
    AS
    SELECT Sno，Sname，Sage FROM Student
        WHERE Sdept= 'IS'
    WITH CHECK OPTION
```

- WITH CHECK OPTION
  透过视图进行增删改操作时，不得破坏视图定义中的谓词条件
  （即子查询中的条件表达式）

- 若一个视图是从单个基本表导出的，并且只是去掉了基本表的某些行和某些列，但保留了基本表的主码，我们称这类视图为行列子集视图。

IS_Student视图就是一个行列子集视图。

[例6] 将学生的学号及他的平均成绩定义为一个视图。（分组视图）

```
CREATE VIEW S_G(Sno，Gavg) AS
    SELECT Sno，AVG(Grade) FROM SC GROUP BY Sno；
```

删除：drop view 视图名. [cascade]

↓

级联删除，将由它导出的视图一起删除

查询、更新与表一样

第4章 数据库安全性

(一) 授权

grant 权限 on 表 to 用户 [with grant option]

用户可传播此权

[例1] 将查询Students表的select权限授给用户U1。
GRANT SELECT ON table Students TO U1;

[例2] 将对Students表和Course表的全部权限授予用户U2和U3。
GRANT ALL PRIVILEGES ON TABLE Students, Course TO U2, U3;

[例3] 将对表SC的查询权限授予所有用户。
GRANT SELECT ON TABLE SC TO PUBLIC;

[例4] 将查询Students表和修改学生学号的权限授给用户U4。
GRANT UPDATE(Sno), SELECT ON TABLE Students TO U4;

[例5] 将SC的INSERT权限授予U5，并允许他将此权限授予其他用户。
GRANT INSERT ON TABLE SC TO U5 WITH GRANT OPTION;

revoke 权限 on 表 from 用户 [cascade | restrict]

(默认) 不允许

[例8] 把用户U4修改学生学号的权限收回。
REVOKE UPDATE(Sno) ON TABLE Students FROM U4;

[例9] 收回所有用户对表SC的查询权限。
REVOKE SELECT ON TABLE SC FROM PUBLIC;

[例10] 把用户U5对SC表的INSERT权限收回。
REVOKE INSERT ON TABLE SC FROM U5 CASCADE;

- 如果系统缺省值为RESTRICT，回收U5的INSERT权限时应该使用CASCADE短语，否则拒绝执行该语句
- 如果U6或U7还从其他用户处获得对SC表的INSERT权限，则他们仍具有此权限，系统只收回直接或间接从U5处获得的权限

若撤销表的 select 权限，并且该表是某视图的基础表，则一并删除视图

(二) 创建用户

用户名 密码

create user 'log'@'localhost' identified by 'password'

创建空角色

创建用户角色. ROLE1

Create ROLE ROLE1

grant role1 To 'log'@'localhost'

通过角色统一授用户权限

[例11] 通过角色来实现权限管理。

步骤如下：

（1）首先创建一个角色 R1

```
CREATE ROLE R1;
```

（2）然后使用GRANT语句，使角色R1拥有Student表的SELECT、UPDATE、INSERT权限

```
GRANT SELECT, UPDATE, INSERT
 ON TABLE Student
 TO R1;
```

（3）将这个角色授予王平，张明，赵玲。使他们具有角色R1所包含的全部权限

```
GRANT R1
   TO 王平,张明,赵玲;
```

（4）可以一次性通过R1来回收王平的这3个权限

```
REVOKE R1
   FROM 王平;
```


例题（续）

[例12] 增加角色的权限

```
GRANT DELETE
ON TABLE Student
TO R1;
```

使角色R1在原来的基础上增加了Student表的DELETE 权限

[例13] 减少角色的权限

```
REVOKE SELECT
ON TABLE Student
FROM R1;
```

使R1减少了SELECT权限

第七章 触发器

```
CREATE TRIGGER <触发器名>
    {BEFORE | AFTER} <触发事件> ON <表名>
    REFERENCING NEW|OLD ROW AS<变量>
    FOR EACH  {ROW | STATEMENT}
    [WHEN <触发条件>]<触发动作体>
```

```
29  SELECT * FROM Student;
30  -- 5.创建INSERT触发器instrg1：在学生表中每新增一名学生，将在班级表class表的总人数自动增加1。
31  CREATE TRIGGER instrg1
32  AFTER INSERT ON Student
33  FOR EACH ROW
34  UPDATE Class SET stotal = stotal + 1 WHERE ClsNO = NEW.ClsNO;
35  -- 6.创建UPDATE触发器update1:当某学生所在班号发生变化时（即调到另一班级后），
36  -- 将其原先所在班级总人数减1，将新调入的班级表class表的总人数增加1。
37  CREATE TRIGGER update1
38  AFTER UPDATE ON Student
39  FOR EACH ROW
40  IF OLD.ClsNO <> NEW.ClsNO THEN
41  UPDATE Class SET stotal = stotal + 1 WHERE ClsNO = NEW.ClsNO;
42  UPDATE Class SET stotal = stotal - 1 WHERE ClsNO = OLD.ClsNO;
43  END IF;
44
```

删除.. drop trigger chggrade on sc

第6章. 关系数据理论.

(一) 公理系统

1> 若 $Y \subseteq X \subseteq U$ $X \to Y$

2> $X \to Y \Rightarrow XZ \to YZ$

3> $X \to Y, Y \to Z \Rightarrow X \to Z$

4> $X \to Y, X \to Z \Rightarrow X \to YZ$

5> $X \to Y, WY \to Z \Rightarrow WX \to Z$

6> $X \to YZ \Rightarrow X \to Y, X \to Z$

实例:设有关系模式R，A、B、C、D、E、F是它的属性子集，R满足下列函数依赖：

F={A→BC， CD→EF}

证明：函数依赖AD→F成立。

证明： A→BC 条件
A→C 分解规则
AD→CD 增广律
CD→EF 条件
AD→EF 传递律
AD→F 分解规则

■ 函数依赖闭包定义

➢ 定义6.12：由被F逻辑蕴涵的函数依赖的全体构成的集合，称为F的闭包，记作F^+。

■ 定义6.13 设F为属性集U上的一组函数依赖，$X \subseteq U$， $X_F^+ = \{ A \mid X \to A$能由$F$根据Armstrong公理导出$\}$，$X_F^+$称为属性集$X$关于函数依赖集$F$ 的闭包。

若 $X_F^+ = U$，X是候选码

例2、 U={A, B, C, D}； F={A→B, BC→D, B→C}；

（1）求A_F^+

解： X(0)=A

∴X(1)=X(0) ∪B=AB, 显然X(1) ≠X(0)。

∴ X(2)=X(1) ∪ C=ABC, 显然X(1) ≠X(2)。

∴ X(3)=X(2) ∪ D=ABCD=U，循环终止。

∴A_F^+=ABCD。

则再在F中找出左边是ABC子集的函数依赖，其结果是：A→B， B→C， BC→D 。则D将加入闭包集中。

2022年4月14日20时51分 数据库系统----第6章 关系数据理论 6.14

（2）求B_F^+。

X(0)=B

X(1)=BCD

X(2)=BCD=X(1),循环终止。所以B_F^+ =BCD。

（3）求C_F^+

X(0)=C

X(1)=C=X(0),循环终止。所以C_F^+ =C。

定义6.15 如果函数依赖集F满足下列条件，则称F为一个极小函数依赖集。亦称为最小依赖集或最小覆盖。

(1) F中任一函数依赖的右部仅含有一个属性。

即F中的函数依赖均不能由F中其他函数依赖导出

(2) F中不存在这样的函数依赖X→A，使得F与F-{X→A}等价。

F中各函数依赖左部均为最小属性集(不存在冗余属性)

(3) F中不存在这样的函数依赖X→A， X有真子集Z使得F-{X→A}∪{Z→A}与F等价。

R<U，F>，U=ABCD，

- 函数依赖集F = {A→BD，AB→C，C→D}。
- 求：F最小函数依赖集
- 解法步骤：
  - 1.将F中的所有函数依赖的右边化为单一属性
  - 2.去掉F中的所有函数依赖左边的冗余属性
  - 3.去掉F中所有冗余的函数依赖

❖（1）将F中的所有函数依赖右边化为单一属性

F={A→BD，AB→C，C→D}

❖ F={ A→B ，A→D ，AB→C ，C→D }

2022年4月14日20时51分 数据库系统---第6章 关系数据理论 6.20

例题图解（2）

不满足一

❖2.去掉F中的所有函数依赖左边的冗余属性

F={A→B，A→D，AB→C，C→D}

$A^+ = \{A, B, C, D\}$ $B^+ = \{B\}$

F={ A→B ，A→D ，A→C ，C→D }

2022年4月14日20时51分 数据库系统---第6章 关系数据理论 6.21

例题图解（3）

❖3.去掉F中所有冗余的函数依赖关系

F={A→B，A→D，A→C，C→D}

F={ A→B ， ，A→C ，C→D }

$A^+ = \{A, B, C, D\}$

F={ A→B ，A→C，C→D }